Universidad Internacional del Ecuador

Carrera: Ingeniería en Software Modalidad En Línea
Materia: PROGRAMACION ORIENTADA A OBJETOS
Estudiante: SALAZAR CADENA JHONNATAN FRANCISCO
Fecha: 1 de marzo del 2026

Evaluación en Contacto con el Docente: El impacto de las nuevas tecnologías en la sociedad

Docente: PALACIOS MOROCHO MILTON RICARDO

Tabla de contenido

Evaluación en Contacto con el Docente: El impacto de las nuevas tecnologías en la sociedad ... 1
Introducción ... 5
Justificación del Proyecto ... 7
Enfoque Social ... 7
Enfoque Técnico ... 8
Justificación del uso de Arquitectura por Capas ... 9
Justificación del uso de Interfaces ... 10
Justificación del uso de SQL Server ... 10
Justificación del uso de API REST ... 11
Justificación del uso de Concurrencia ... 11
Justificación de las Pruebas Automatizadas ... 12
Conclusión de la Justificación ... 12
Arquitectura General del Sistema ... 12
Enfoque Arquitectónico Adoptado ... 13
Flujo de Ejecución del Sistema ... 14
Principios de Diseño Aplicados ... 14
Separación de Responsabilidades ... 15
Inversión de Dependencias ... 15
Justificación de la Modularización del Proyecto ... 16
Relación entre Arquitectura y Escalabilidad ... 17
Diagrama de Clases ... 17
Componentes Principales ... 18
Clase Libro (Modelo de Dominio) ... 18
Interfaz BookRepository (Contrato de Persistencia) ... 18
Clase SQLServerStore (Implementación Concreta) ... 18
Clase Catalog (Lógica de Negocio) ... 19
Clase APIHandler (Capa de Exposición) ... 19
Relaciones UML Relevantes ... 19
Interpretación Arquitectónica ... 19
Descripción Técnica del Sistema ... 20
Estructura del Proyecto ... 20
Carpeta cmd/ ... 20
Carpeta internal/ ... 21
internal/model/ ... 22
internal/catalog/ ... 22

internal/storage/ ... 23
internal/api/ ... 24
Carpeta web/ ... 25
Descripción de Model ... 26
Rol del Modelo dentro de la Arquitectura ... 26
Definición Estructural del Modelo ... 26
Decisiones de Diseño del Modelo ... 27
Relación del Modelo con la Base de Datos ... 27
Independencia del Modelo frente a Infraestructura ... 28
Evaluación Técnica del Modelo ... 29
Descripción de Repository ... 29
Rol del Repository dentro de la Arquitectura ... 29
Interfaz BookRepository (Contrato de Persistencia) ... 30
Justificación del Diseño Basado en Interfaces ... 30
Implementación Concreta: SQLServerStore ... 31
Manejo de Errores en el Repository ... 31
Descripción de Catalog ... 32
Rol del Catalog dentro del Sistema ... 32
Diseño Interno y Control de Responsabilidades ... 32
Métodos Principales del Catálogo ... 33
AddBook (Crear un Libro) ... 33
GetByID (Consultar un Libro por Identificador) ... 33
SearchBooks (Búsqueda) ... 33
UpdateBook (Actualizar) ... 34
DeleteBook (Eliminar) ... 34
ToggleAvailability (Cambiar Disponibilidad) ... 35
Descripción de API ... 35
Concurrencia ... 36
Base de Datos ... 37
Pruebas Realizadas ... 37
Pruebas Unitarias ... 37
Pruebas de Integración ... 39
Pruebas de Aceptación ... 39
Dificultades Encontradas ... 40
Configuración y Conectividad con SQL Server ... 41
Manejo Correcto de Rutas y Servidor HTTP ... 41
Sincronización y Concurrencia ... 41

Separación Correcta de Responsabilidades 42
Corrección de Pruebas Unitarias 42
Reflexión Técnica 42
Conclusiones 42
Bibliografía 43
URL web repositorio de Github 44

Introducción

El presente documento corresponde al **Informe Técnico – Manual del Programador** del proyecto **Sistema de Gestión de Libros Electrónicos**, desarrollado como Proyecto Final Integrador de la asignatura **Programación Orientada a Objetos (POO)**. Este manual tiene como finalidad documentar de manera estructurada y profesional la arquitectura, los componentes internos, el diseño por capas, los mecanismos de persistencia en base de datos relacional, la exposición de servicios web REST, la implementación de concurrencia y la estrategia de pruebas aplicadas al sistema.

El sistema implementa una aplicación web funcional orientada a la administración de un catálogo de libros electrónicos, permitiendo realizar operaciones completas de tipo **CRUD (Create, Read, Update, Delete)**, así como funcionalidades adicionales de búsqueda y generación de estadísticas. Estas capacidades se exponen mediante una **API REST**, diseñada bajo estándares HTTP y consumida por una interfaz web que actúa como cliente del backend.

Desde una perspectiva de ingeniería de software, este proyecto no se limita a demostrar funcionamiento operativo, sino que busca evidenciar principios fundamentales de diseño tales como:

- **Separación de responsabilidades**
- **Bajo acoplamiento**
- **Alta cohesión**
- **Programación orientada a interfaces**
- **Arquitectura modular por capas**
- **Control de concurrencia en entornos web**
- **Persistencia robusta en base de datos relacional**

El desarrollo fue realizado utilizando **Go (Golang)** como lenguaje principal, aprovechando su eficiencia en aplicaciones backend, su estructura modular basada en paquetes y su soporte nativo para concurrencia. La persistencia de los datos se implementa en **SQL Server**, un motor de base de datos relacional de nivel empresarial, administrado mediante SQL Server Management Studio (SSMS). Esta decisión técnica incrementa el realismo del proyecto al incorporar aspectos propios de entornos productivos, como:

- Gestión de conexiones.
- Modelado tabular.
- Operaciones persistentes.
- Integridad y consistencia de datos.
- Interacción directa con un sistema gestor de base de datos real.

El sistema adopta una arquitectura por capas claramente definida, organizada en módulos como:

- model
- repository
- storage
- catalog
- api
- web

Cada capa cumple un rol específico dentro del flujo de ejecución. El modelo representa la entidad principal del dominio (Libro); el repositorio define el contrato de persistencia mediante una interfaz; la capa de almacenamiento implementa dicho contrato utilizando SQL Server; el catálogo centraliza la lógica de negocio; la API expone las funcionalidades como servicios REST; y la capa web consume estos servicios desde el navegador. Esta organización estructural permite mantener independencia entre dominio, infraestructura y presentación, facilitando la mantenibilidad y evolución futura del sistema.

Un aspecto técnico relevante del proyecto es el manejo explícito de **concurrencia**. Dado que el servidor HTTP puede recibir múltiples solicitudes simultáneamente, se implementa el uso de primitivas de sincronización como **sync.RWMutex** para proteger operaciones críticas de lectura y escritura. Esto garantiza consistencia de datos y previene condiciones de carrera, aspecto esencial en aplicaciones backend concurrentes.

Asimismo, el proyecto incorpora un enfoque de validación mediante **pruebas unitarias**, pruebas de integración y pruebas de aceptación, lo cual fortalece la confiabilidad del sistema y documenta su comportamiento esperado. Este enfoque no solo cumple con requisitos académicos, sino que refleja buenas prácticas de desarrollo profesional.

Finalmente, el documento incluye una sección de **Visualización del Futuro**, en la cual se analizan posibles mejoras como migración a microservicios, autenticación basada en tokens, despliegue en contenedores y adopción de infraestructura en la nube. Esta proyección estratégica demuestra que el sistema fue diseñado con capacidad de evolución y escalabilidad.

En conclusión, este manual no es únicamente una descripción funcional del software, sino una documentación técnica integral que expone las decisiones de diseño, los principios aplicados y la arquitectura implementada, proporcionando una base sólida para su evaluación académica y su posible expansión futura.

Justificación del Proyecto

Enfoque Social

En la actualidad, la transformación digital ha redefinido la manera en que las organizaciones gestionan información, recursos y conocimiento. En el ámbito académico y organizacional, la administración de recursos digitales —como libros electrónicos— exige sistemas estructurados que permitan almacenamiento seguro, acceso eficiente y control centralizado. La ausencia de herramientas adecuadas provoca desorganización, duplicidad de información, pérdida de trazabilidad y baja eficiencia operativa.

El **Sistema de Gestión de Libros Electrónicos** surge como respuesta a esta problemática, proponiendo una solución tecnológica que digitaliza, organiza y sistematiza la administración de un catálogo de libros electrónicos mediante una aplicación web estructurada y persistente.

Desde una perspectiva social, el proyecto aporta los siguientes beneficios:

- **Optimización del acceso al conocimiento digital.**
  Permite consultar información organizada de manera estructurada, facilitando la búsqueda y recuperación eficiente de recursos.
- **Reducción de errores humanos.**
  La automatización de operaciones CRUD disminuye inconsistencias propias de la gestión manual.
- **Transparencia y control.**
  El sistema incorpora mecanismos de actualización y verificación del estado de disponibilidad, permitiendo un seguimiento claro del catálogo.
- **Digitalización alineada con tendencias actuales.**
  Las instituciones educativas y organizaciones modernas requieren soluciones basadas en arquitecturas web y bases de datos robustas, alejándose de registros informales o desestructurados.

Asimismo, el proyecto contribuye al desarrollo de competencias profesionales del desarrollador, al simular un entorno real de trabajo backend con persistencia en SQL Server, exposición de servicios REST y manejo de concurrencia, lo cual trasciende ejercicios académicos básicos y se aproxima a escenarios empresariales reales.

En consecuencia, el impacto social del sistema no radica únicamente en su funcionalidad, sino en la adopción de buenas prácticas que promueven digitalización organizada, escalabilidad y sostenibilidad tecnológica.

Enfoque Técnico

Desde el punto de vista técnico, el proyecto fue diseñado para demostrar dominio integral de conceptos avanzados de ingeniería de software y programación orientada a objetos. No se trata de un simple CRUD aislado, sino de una arquitectura estructurada que evidencia principios de diseño profesional para demostrar dominio en:

- **Programación Orientada a Objetos (POO)**
- **Arquitectura por capas**
- **Diseño basado en interfaces**
- **Servicios Web REST**
- **Serialización JSON**
- **Persistencia en SQL Server**
- **Concurrencia en entornos web**
- **Pruebas automatizadas**

```
Problems    Output    Debug Console    Terminal    Ports

PS C:\Users\jhonn\OneDrive\Documents\Antigravity\Sistema_Gestion_Libros_Electronicos> go test ./... -v
?       sistema_libros/cmd/app  [no test files]
?       sistema_libros/cmd/server       [no test files]
?       sistema_libros/internal/api     [no test files]
=== RUN   TestGetByID_Found
--- PASS: TestGetByID_Found (0.00s)
=== RUN   TestGetByID_NotFound
--- PASS: TestGetByID_NotFound (0.00s)
=== RUN   TestSearchBooks_ByTitle
--- PASS: TestSearchBooks_ByTitle (0.00s)
=== RUN   TestSearchBooks_ByAuthor
--- PASS: TestSearchBooks_ByAuthor (0.00s)
=== RUN   TestSearchBooks_ByCategory
--- PASS: TestSearchBooks_ByCategory (0.00s)
=== RUN   TestAddBook_AssignsNewID
--- PASS: TestAddBook_AssignsNewID (0.00s)
=== RUN   TestUpdateBook_UpdatesFields
--- PASS: TestUpdateBook_UpdatesFields (0.00s)
=== RUN   TestUpdateBook_NotFound
--- PASS: TestUpdateBook_NotFound (0.00s)
=== RUN   TestDeleteBook_RemovesItem
--- PASS: TestDeleteBook_RemovesItem (0.00s)
=== RUN   TestDeleteBook_NotFound
--- PASS: TestDeleteBook_NotFound (0.00s)
=== RUN   TestToggleAvailability_Toggles
--- PASS: TestToggleAvailability_Toggles (0.00s)
=== RUN   TestToggleAvailability_NotFound
--- PASS: TestToggleAvailability_NotFound (0.00s)
PASS
ok      sistema_libros/internal/catalog (cached)
?       sistema_libros/internal/model   [no test files]
?       sistema_libros/internal/storage [no test files]
?       sistema_libros/internal/ui      [no test files]
PS C:\Users\jhonn\OneDrive\Documents\Antigravity\Sistema_Gestion_Libros_Electronicos>
```

Ilustración 1 Ejecución de pruebas (go test)

Justificación del uso de Arquitectura por Capas

El sistema adopta una arquitectura modular organizada en capas claramente diferenciadas:

- **Model (Dominio)**
- **Repository (Contrato de persistencia)**
- **Storage (Implementación concreta)**
- **Catalog (Lógica de negocio)**
- **API (Exposición REST)**
- **Web (Cliente)**

Esta separación responde a principios fundamentales como:

- **Single Responsibility Principle**
- **Inversión de Dependencias**
- **Bajo Acoplamiento**
- **Alta Cohesión**

La arquitectura por capas permite que cada componente tenga responsabilidades bien definidas, evitando que la lógica de negocio dependa directamente de detalles de infraestructura. Por ejemplo, el catálogo no conoce cómo se almacenan los datos; únicamente interactúa con una interfaz que define operaciones abstractas.

Esta decisión técnica incrementa la mantenibilidad, facilita pruebas unitarias y prepara el sistema para evolución futura.

Cada capa tiene un rol específico:

Capa	Responsabilidad
Model	Representación del dominio
Repository	Contrato de persistencia
Storage	Implementación concreta de almacenamiento
Catalog	Lógica de negocio
API	Exposición de servicios REST
Web	Cliente consumidor

Este diseño permite:

- Reemplazar el motor de persistencia sin modificar la lógica de negocio.
- Probar componentes de forma aislada.
- Escalar el sistema hacia microservicios en el futuro.

Justificación del uso de Interfaces

La implementación de la interfaz BookRepository representa una aplicación directa del principio:

“Programar hacia abstracciones, no hacia implementaciones concretas”.

Al definir un contrato de persistencia independiente de la tecnología subyacente, el sistema logra:

- Desacoplar la lógica de negocio del motor de base de datos.
- Permitir múltiples implementaciones de almacenamiento.
- Facilitar pruebas mediante repositorios simulados.
- Preparar el sistema para migraciones futuras sin modificar la capa de negocio.

Esta decisión fortalece la extensibilidad y demuestra comprensión profunda de POO aplicada en contextos backend.

Justificación del uso de SQL Server

La persistencia de datos se implementa utilizando **SQL Server**, motor de base de datos relacional ampliamente utilizado en entornos empresariales.

Esta elección se fundamenta en:

- Robustez y estabilidad.
- Soporte para transacciones.
- Integridad referencial.
- Escalabilidad.
- Administración mediante SSMS.
-

La utilización de una base de datos relacional real eleva la complejidad técnica del proyecto, ya que implica:

- Gestión de conexiones.
- Modelado tabular.
- Mapeo entre estructuras del dominio y tablas relacionales.
- Manejo de errores de persistencia.

Esta decisión posiciona el proyecto en un nivel profesional, alejándolo de implementaciones simplificadas basadas únicamente en almacenamiento temporal.

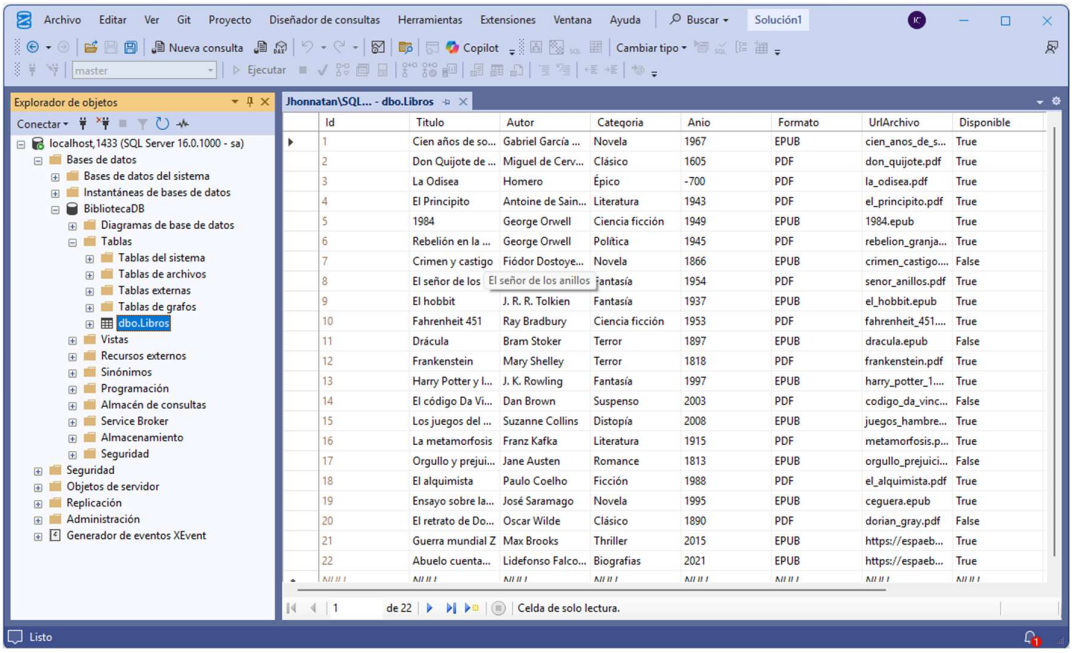

Ilustración 2 Base de datos en SQL Server (SSMS)

Justificación del uso de API REST

La exposición del sistema mediante una API REST permite desacoplar frontend y backend, cumpliendo con estándares modernos de desarrollo web.

La API implementa múltiples endpoints que gestionan recursos bajo convenciones HTTP adecuadas (GET, POST, PUT, DELETE, PATCH), lo cual garantiza:

- Claridad semántica.
- Interoperabilidad.
- Posibilidad de múltiples consumidores.
- Escalabilidad futura hacia arquitecturas distribuidas.

El uso de servicios REST no solo cumple con un requisito académico, sino que introduce una estructura profesional alineada con arquitecturas empresariales modernas.

Justificación del uso de Concurrencia

Un servidor web opera inherentemente en un entorno concurrente. Cada solicitud HTTP puede ejecutarse en paralelo, lo que genera riesgos de inconsistencia si no se controlan adecuadamente los accesos a recursos compartidos.

Por ello, el sistema implementa:

- **sync.RWMutex**

- Bloqueos diferenciados de lectura y escritura.
- Protección de secciones críticas.

Esta decisión garantiza:

- Prevención de condiciones de carrera.
- Consistencia de estado.
- Seguridad en acceso simultáneo.

La inclusión explícita de mecanismos de sincronización demuestra comprensión del comportamiento concurrente en aplicaciones backend y eleva el nivel técnico del proyecto.

Justificación de las Pruebas Automatizadas

El proyecto incluye:

- Pruebas unitarias (go test).
- Validación de lógica del catálogo.
- Verificación de operaciones CRUD.

Las pruebas permiten:

- Validar funcionamiento correcto.
- Detectar regresiones.
- Documentar comportamiento esperado.

Conclusión de la Justificación

El Sistema de Gestión de Libros Electrónicos no es únicamente una aplicación CRUD, sino una demostración estructurada de arquitectura profesional aplicada en un entorno académico.

El proyecto cumple con:

- Requisitos funcionales.
- Requisitos técnicos.
- Buenas prácticas de ingeniería de software.
- Preparación para evolución futura.

Por tanto, se justifica tanto desde un enfoque social (digitalización estructurada del conocimiento) como desde un enfoque técnico (aplicación rigurosa de principios de diseño y arquitectura).

Arquitectura General del Sistema

La arquitectura del **Sistema de Gestión de Libros Electrónicos** fue diseñada bajo un enfoque estructurado por capas, con el objetivo de garantizar separación de responsabilidades, mantenibilidad, extensibilidad y claridad conceptual. Esta decisión no responde únicamente a una organización estética del código, sino a la necesidad de

aplicar principios formales de ingeniería de software que permitan que el sistema sea comprensible, escalable y técnicamente defendible.

A diferencia de una implementación monolítica donde las responsabilidades se mezclan (interfaz, lógica, persistencia en un mismo archivo), este sistema se estructura en módulos claramente definidos, cada uno con un rol específico dentro del flujo de ejecución.

Enfoque Arquitectónico Adoptado

El sistema adopta una **arquitectura en capas desacopladas**, donde cada módulo depende únicamente de abstracciones y no de implementaciones concretas. Este patrón permite controlar el acoplamiento y garantizar que los cambios en infraestructura no afecten la lógica del dominio.

Las capas principales son:

Capa	Responsabilidad Principal
Web	Interfaz cliente (HTML)
API	Exposición de endpoints REST
Catalog	Lógica de negocio
Repository	Contrato de persistencia
Storage	Implementación en SQL Server
Model	Representación del dominio

Esta estructura responde a un principio fundamental:

El dominio del negocio no debe depender de la infraestructura.

En consecuencia, el modelo Libro y la lógica del catálogo no conocen detalles específicos sobre SQL Server; únicamente interactúan con una interfaz que define operaciones abstractas.

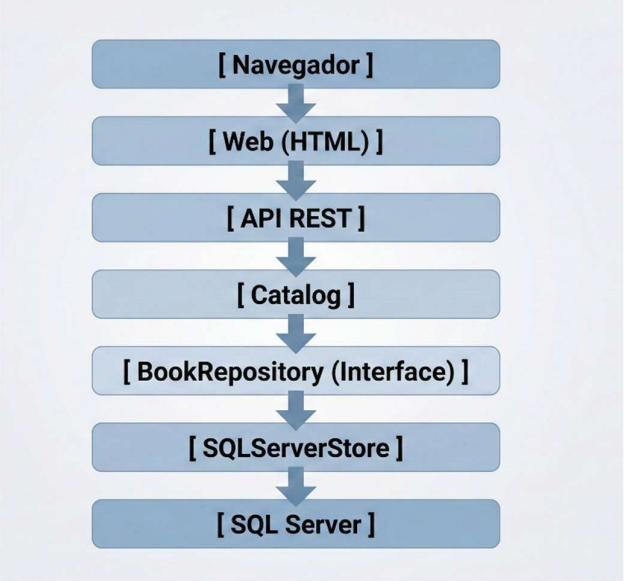

Ilustración 3 Diagrama en bloques por capas

Flujo de Ejecución del Sistema

Desde el punto de vista operativo, el sistema sigue un flujo de solicitud-respuesta estructurado:

1. El usuario interactúa con la interfaz web.
2. El navegador envía una solicitud HTTP hacia la API.
3. El servidor procesa la solicitud a través del router.
4. El handler correspondiente invoca métodos del catálogo.
5. El catálogo ejecuta lógica de negocio y delega persistencia al repositorio.
6. El repositorio invoca la implementación concreta (SQLServerStore).
7. SQL Server ejecuta la operación y devuelve resultados.
8. La respuesta se serializa y retorna al cliente.

Este flujo garantiza que cada capa cumpla una responsabilidad específica y que el procesamiento sea predecible.

Desde una perspectiva técnica, este diseño permite:

- Insertar validaciones en la capa de negocio sin afectar la API.
- Modificar el motor de base de datos sin alterar el catálogo.
- Agregar nuevas rutas sin modificar el modelo.
- Probar el catálogo sin necesidad de ejecutar el servidor web.

Principios de Diseño Aplicados

La arquitectura no es accidental; está sustentada en principios formales:

Separación de Responsabilidades

Cada módulo tiene un propósito único:

- El modelo define estructura de datos.
- El catálogo contiene reglas.
- La API gestiona transporte HTTP.
- La persistencia interactúa con la base de datos.

Esto evita duplicidad de lógica y facilita mantenimiento.

Inversión de Dependencias

La capa de negocio depende de la interfaz BookRepository, no de SQLServerStore. Este diseño permite intercambiar implementaciones sin modificar el catálogo.

Representación conceptual:

Catalog → BookRepository (Interface)

BookRepository ← SQLServerStore (Implementación)

Esta relación demuestra aplicación práctica de POO avanzada.

internal > storage > sqlserver_store.go > ...

```go
// SQLServerStore implementa model.BookRepository usando SQL Server.
// Mantiene tu arquitectura actual: Load() trae todo a memoria y Save() guarda todo.
type SQLServerStore struct {
    db *sql.DB
}

// NewSQLServerStore crea el store y valida conexión.
func NewSQLServerStore(connString string) (*SQLServerStore, error) {
    db, err := sql.Open("sqlserver", connString)
    if err != nil {
        return nil, err
    }

    ctx, cancel := context.WithTimeout(context.Background(), 5*time.Second)
    defer cancel()

    if err := db.PingContext(ctx); err != nil {
        _ = db.Close()
        return nil, err
    }

    return &SQLServerStore{db: db}, nil
}

func (s *SQLServerStore) Close() error {
    return s.db.Close()
}

// Load trae todos los libros desde dbo.Libros.
func (s *SQLServerStore) Load() ([]model.Libro, error) {
    ctx, cancel := context.WithTimeout(context.Background(), 5*time.Second)
    defer cancel()

    rows, err := s.db.QueryContext(ctx, `
        SELECT Id, Titulo, Autor, Categoria, Anio, Formato, UrlArchivo, Disponible
        FROM dbo.Libros
        ORDER BY Id;
```

Ilustración 4 Estructura SQLServerStore

Justificación de la Modularización del Proyecto

La estructura física del proyecto refuerza la arquitectura lógica:

cmd/

  app/

  server/

internal/

  model/

  catalog/

  storage/

  api/

web/

Esta organización permite:

- Separar punto de entrada (main) del núcleo del sistema.
- Aislar lógica interna dentro de internal.
- Evitar dependencias circulares.
- Facilitar empaquetado y despliegue.

Además, el uso de internal protege componentes para que no sean importados externamente, reforzando encapsulamiento.

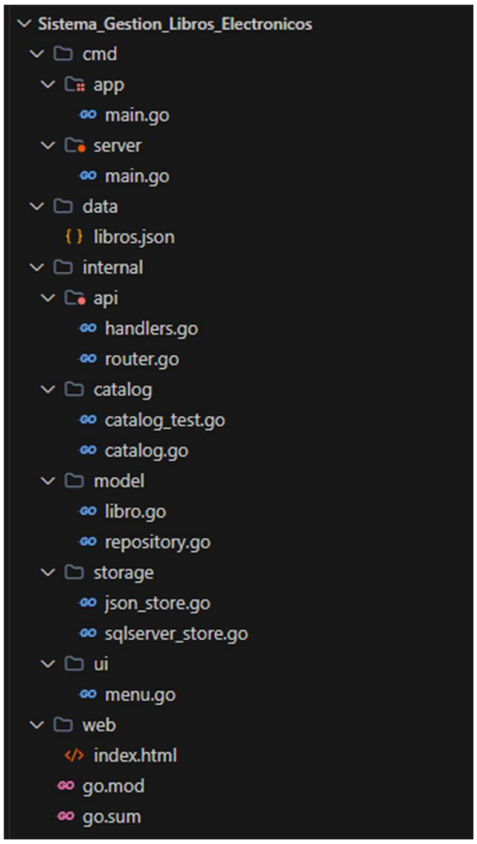

Ilustración 5 Arbol de Carpetas

Relación entre Arquitectura y Escalabilidad

La arquitectura adoptada no solo organiza el presente, sino que habilita crecimiento futuro. Al separar capas y depender de interfaces:

- Es posible migrar a microservicios.
- Se puede incorporar autenticación sin alterar el dominio.
- Puede añadirse cache intermedio.
- Puede implementarse un ORM sin cambiar la lógica de negocio.

Este diseño demuestra visión estratégica y madurez técnica.

Diagrama de Clases

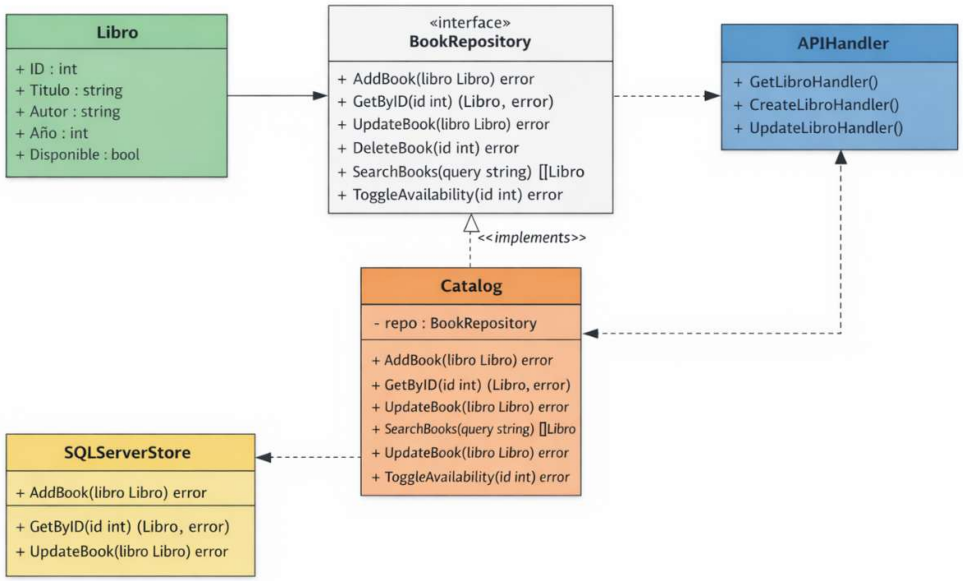

Ilustración 6 Diagrama de Clases del Sistema de Gestión de Libros Electrónicos.

El Diagrama de Clases del **Sistema de Gestión de Libros Electrónicos** representa la estructura estática del sistema y evidencia la aplicación formal de los principios de **Programación Orientada a Objetos (POO)**.

Este diagrama no solo muestra entidades, sino que documenta la separación de responsabilidades, el uso de interfaces como contratos y la relación entre las distintas capas del sistema.

Componentes Principales

El sistema está compuesto por cinco elementos estructurales clave:

Clase Libro (Modelo de Dominio)

Representa la entidad principal del sistema. Contiene los atributos que definen el estado del recurso gestionado:

- ID : int
- Titulo : string
- Autor : string
- Año : int
- Disponible : bool

Su función es servir como modelo transversal entre las capas de negocio y persistencia.

Interfaz BookRepository (Contrato de Persistencia)

Define las operaciones abstractas que permiten gestionar libros sin depender de una implementación específica.

Métodos principales:

- AddBook()
- GetByID()
- UpdateBook()
- DeleteBook()
- SearchBooks()
- ToggleAvailability()

Su propósito es desacoplar la lógica del negocio de la infraestructura.

Clase SQLServerStore (Implementación Concreta)

Implementa la interfaz BookRepository y encapsula el acceso a la base de datos SQL Server.

Responsabilidades:

- Ejecutar sentencias SQL.
- Mapear registros a estructuras Libro.
- Manejar errores de persistencia.

En el diagrama se representa mediante una relación de **realización (implements)** hacia la interfaz.

Clase Catalog (Lógica de Negocio)

Actúa como orquestador del sistema.

Contiene:

- Referencia a BookRepository.
- Implementación de reglas de negocio.
- Coordinación de operaciones CRUD.

El catálogo depende de la interfaz, no de la implementación concreta, aplicando el principio de **Inversión de Dependencias**.

Clase APIHandler (Capa de Exposición)

Encargada de:

- Recibir solicitudes HTTP.
- Invocar métodos del catálogo.
- Retornar respuestas adecuadas.

No contiene lógica de negocio ni acceso directo a base de datos.

Relaciones UML Relevantes

El diagrama evidencia las siguientes relaciones:

- Catalog → BookRepository (Dependencia hacia abstracción)
- SQLServerStore → BookRepository (Implementación de interfaz)
- APIHandler → Catalog (Uso de lógica de negocio)
- Catalog → Libro (Uso del modelo de dominio)

Estas relaciones garantizan bajo acoplamiento y alta cohesión.

Interpretación Arquitectónica

El Diagrama de Clases confirma que:

- La API no accede directamente a la base de datos.
- La lógica de negocio no depende de SQL Server.
- El modelo es independiente de la infraestructura.
- La persistencia está encapsulada.

En conjunto, el UML demuestra que el sistema fue diseñado siguiendo principios formales de arquitectura por capas y programación orientada a objetos.

Descripción Técnica del Sistema

Estructura del Proyecto

La estructura del proyecto fue diseñada siguiendo un enfoque modular alineado con principios de arquitectura limpia y separación de responsabilidades. No se trata simplemente de una organización por carpetas, sino de una distribución intencional que refleja la arquitectura lógica del sistema.

El proyecto se organiza de la siguiente manera:

```
Sistema_Gestion_Libros_Electronicos/
│
├── cmd/
│   ├── app/
│   └── server/
│
├── internal/
│   ├── model/
│   ├── catalog/
│   ├── storage/
│   ├── api/
│
├── web/
│   └── index.html
```

Carpeta cmd/

La carpeta cmd contiene los puntos de entrada del sistema. En proyectos Go profesionales, es una práctica recomendada separar el ejecutable principal del núcleo lógico del sistema.

cmd/app/

Contiene la versión de ejecución en modo consola.

Su propósito es permitir pruebas directas del sistema sin necesidad de iniciar el servidor HTTP.

Desde el punto de vista técnico, esta separación:

- Permite ejecutar pruebas manuales del catálogo.
- Facilita depuración independiente del entorno web.
- Demuestra modularidad del sistema.

cmd/server/

Contiene el punto de entrada del servidor web (main.go).

Aquí se realiza:

- Inicialización de la conexión a SQL Server.
- Creación del repositorio concreto.
- Inyección de dependencias en el catálogo.
- Configuración del router.
- Inicio del servidor HTTP.

Este archivo representa la capa de composición del sistema, donde se ensamblan las piezas.

```
cmd > server > main.go > ...
13
14  func main() {
15      // DB
16      conn := os.Getenv("DB_CONN")
17      if conn == "" {
18          conn = "server=localhost,1433;user id=sa;password=TuNuevaClave123!;database=BibliotecaDB;encrypt=true;trustserverc
19      }
20
21      repo, err := storage.NewSQLServerStore(conn)
22      if err != nil {
23          log.Fatalf("Error conectando a SQL Server: %v", err)
24      }
25      defer repo.Close()
26
27      // API mux (tus rutas /api y /health)
28      s := &api.Server{Repo: repo}
29      apiMux := api.NewMux(s)
30
31      // Mux principal
32      rootMux := http.NewServeMux()
33
34      // 1) API
35      rootMux.Handle("/api/", apiMux)
36      rootMux.Handle("/health", apiMux)
37
38      // 2) Frontend: servir carpeta web/
39      //    IMPORTANTE: "web" debe estar en la raíz del proyecto (como en tu captura)
40      rootMux.Handle("/", http.FileServer(http.Dir("web")))
41
42      // Logging
43      handler := http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
44          start := time.Now()
45          rootMux.ServeHTTP(w, r)
46          fmt.Printf("%s %s (%s)\n", r.Method, r.URL.Path, time.Since(start))
47      })
48
49      addr := ":8080"
50      fmt.Println("====================================")
51      fmt.Println("Servidor: http://localhost" + addr + "/")
```

Ilustración 7 Punto de entrada del servidor web (main.go).

Carpeta internal/

La carpeta internal contiene el núcleo del sistema. En Go, el uso de internal restringe la importación de estos paquetes desde fuera del módulo, reforzando el encapsulamiento. Esta decisión arquitectónica tiene implicaciones importantes:

- Evita dependencias externas indebidas.
- Refuerza la separación entre núcleo e interfaz pública.
- Mejora mantenibilidad.

Dentro de internal se encuentran las capas principales.

internal/model/

Contiene la definición del modelo Libro.

Su función es representar el dominio sin depender de infraestructura ni transporte HTTP.

Principios aplicados:

- Modelo puro.
- Independencia tecnológica.
- Representación coherente del estado del sistema.

internal > model > libro.go > ...

```go
package model

import "fmt"

type Libro struct {
    ID         int    `json:"id"`
    Titulo     string `json:"titulo"`
    Autor      string `json:"autor"`
    Categoria  string `json:"categoria"`
    Anio       int    `json:"anio"`
    Formato    string `json:"formato"`
    URLArchivo string `json:"urlArchivo"`
    Disponible bool   `json:"disponible"`
}

func (l Libro) String() string {
    estado := "No disponible"
    if l.Disponible {
        estado = "Disponible"
    }
    return fmt.Sprintf(
        "ID: %d | Título: %s | Autor: %s | Categoría: %s | Año: %d | Formato: %s | Archivo: %s | Estado: %s",
        l.ID, l.Titulo, l.Autor, l.Categoria, l.Anio, l.Formato, l.URLArchivo, estado,
    )
}
```

Ctrl+I Ctrl+L for Agent

Ilustración 8 Archivo Libro

internal/catalog/

Esta carpeta contiene la lógica de negocio.

Es el núcleo funcional del sistema.

Responsabilidades:

- Validar datos antes de persistir.
- Orquestar llamadas al repositorio.
- Implementar reglas específicas.
- Garantizar coherencia lógica.

Desde una perspectiva arquitectónica, esta capa:

- No conoce detalles de SQL.
- No maneja HTTP.
- No renderiza vistas.

Solo gestiona comportamiento del dominio.

```
internal > catalog > catalog.go > NextID
9
10  var (
11      ErrNotFound    = errors.New("libro no encontrado")
12      ErrInvalidData = errors.New("datos inválidos")
13  )
14
15  func NextID(books []model.Libro) int {
16      maxID := 0
17      for _, b := range books {
18          if b.ID > maxID {
19              maxID = b.ID
20          }
21      }
22      return maxID + 1
23  }
24
25  func AddBook(books []model.Libro, nuevo model.Libro) ([]model.Libro, error) {
26      nuevo.Titulo = strings.TrimSpace(nuevo.Titulo)
27      nuevo.Autor = strings.TrimSpace(nuevo.Autor)
28      nuevo.Categoria = strings.TrimSpace(nuevo.Categoria)
29      nuevo.Formato = strings.TrimSpace(nuevo.Formato)
30      nuevo.URLArchivo = strings.TrimSpace(nuevo.URLArchivo)
31
32      if nuevo.Titulo == "" || nuevo.Autor == "" {
33          return books, ErrInvalidData
34      }
```

Ilustración 9 Archivo Catalog

internal/storage/

Contiene la implementación concreta del repositorio.

En este proyecto se incluye:

- SQLServerStore

Esta capa encapsula:

- Conexión a base de datos.
- Consultas SQL.
- Manejo de errores de persistencia.
- Mapeo entre filas y estructuras Libro.

Es importante destacar que cualquier modificación en la base de datos impacta únicamente esta capa, no el catálogo ni la API.

```
internal > storage > sqlserver_store.go > (*SQLServerStore).Close
16  type SQLServerStore struct {
17      db *sql.DB
18  }
19
20  // NewSQLServerStore crea el store y valida conexión.
21  func NewSQLServerStore(connString string) (*SQLServerStore, error) {
22      db, err := sql.Open("sqlserver", connString)
23      if err != nil {
24          return nil, err
25      }
26
27      ctx, cancel := context.WithTimeout(context.Background(), 5*time.Second)
28      defer cancel()
29
30      if err := db.PingContext(ctx); err != nil {
31          _ = db.Close()
32          return nil, err
33      }
34
35      return &SQLServerStore{db: db}, nil
36  }
37
38  func (s *SQLServerStore) Close() error {
39      return s.db.Close()
```

Ilustración 10 Archivo SQLServerStorage

internal/api/

Contiene los handlers HTTP y configuración de rutas.

Responsabilidades:

- Recibir solicitudes.
- Validar parámetros HTTP.
- Invocar métodos del catálogo.
- Retornar códigos de estado adecuados.

No implementa reglas de negocio ni lógica de persistencia.

Esta separación garantiza que el transporte (HTTP) esté desacoplado del dominio.

```go
internal > api > router.go > NewMux
package api

import "net/http"

// NewMux configura las rutas (8+ servicios) usando el ServeMux de Go 1.22+.
// Patrones con método + path y variables: "GET /api/libros/{id}".
func NewMux(s *Server) *http.ServeMux {
    mux := http.NewServeMux()

    // Salud (útil para demo)
    mux.HandleFunc("GET /health", s.handleHealth)

    // 1) GET /api/libros
    mux.HandleFunc("GET /api/libros", s.handleListBooks)

    // 7) GET /api/libros/buscar?q=texto
    mux.HandleFunc("GET /api/libros/buscar", s.handleSearchBooks)

    // 8) GET /api/estadisticas
    mux.HandleFunc("GET /api/estadisticas", s.handleStats)

```

Ilustración 11 Archivo Router

Carpeta web/

Contiene el archivo index.html, que actúa como cliente del backend.

Desde el punto de vista arquitectónico:

- No contiene lógica de negocio.
- Consume endpoints REST.
- Permite validar funcionamiento extremo a extremo.

Este componente demuestra la separación frontend-backend.

```html
web > index.html > ...
<!doctype html>
<html lang="es">
<head>
  <meta charset="utf-8" />
  <meta name="viewport" content="width=device-width,initial-scale=1" />
  <title>Sistema de Gestión de E-Books</title>
  <style>
    body{font-family:system-ui,Arial;max-width:1000px;margin:24px auto;padding:0 12px}
    h1{margin:0 0 12px}
    .row{display:flex;gap:8px;flex-wrap:wrap;align-items:center}
    input,select,button{padding:10px;border:1px solid #ccc;border-radius:8px}
    button{cursor:pointer}
    table{width:100%;border-collapse:collapse;margin-top:12px}
    th,td{border-bottom:1px solid #eee;padding:10px;text-align:left;vertical-align:top}
    .badge{padding:2px 8px;border-radius:999px;border:1px solid #ddd;font-size:12px}
    .ok{background:#eaffea}
    .no{background:#ffeaea}
    .card{border:1px solid #eee;border-radius:12px;padding:12px;margin:12px 0}
    .muted{color:#666}
    .actions button{margin-right:6px}
  </style>
</head>
<body>
  <h1> Sistema de Gestión de E-Books</h1>
  <p class="muted">Frontend simple (HTML) consumiendo tu API Go en <code>/api</code>.</p>
```

Ilustración 12 Codigo html

Descripción de Model

Rol del Modelo dentro de la Arquitectura

El modelo Libro constituye la representación formal del **dominio principal del sistema**. En una arquitectura por capas correctamente implementada, el modelo no es simplemente una estructura de datos, sino la expresión conceptual del recurso que el sistema administra.

En el **Sistema de Gestión de Libros Electrónicos**, Libro es la entidad central alrededor de la cual giran todas las operaciones: creación, consulta, actualización, eliminación, búsqueda y cambio de disponibilidad. Desde el punto de vista arquitectónico, esta estructura cumple una función transversal, ya que es utilizada por:

- La capa de persistencia (para mapear registros SQL).
- La capa de negocio (Catalog) para aplicar reglas.
- La API para enviar y recibir información.
- El frontend como representación estructurada del recurso.

El modelo, sin embargo, **no depende de ninguna de estas capas**, lo que refuerza el principio de independencia del dominio frente a infraestructura y transporte.

Definición Estructural del Modelo

El modelo Libro está compuesto por los siguientes atributos fundamentales:

```
type Libro struct {
    ID        int
    Titulo    string
    Autor     string
    Año       int
    Disponible bool
}
```

Análisis Técnico de Atributos

Atributo	Tipo	Justificación Técnica
ID	int	Identificador único del libro (clave primaria en BD).
Titulo	string	Representa el nombre del recurso.
Autor	string	Permite clasificación y búsqueda.
Año	int	Información estructurada para consultas.
Disponible	bool	Control de estado lógico del recurso.

Decisiones de Diseño del Modelo

1 Uso de Tipos Primitivos

Se utilizan tipos primitivos (int, string, bool) por las siguientes razones:

- Compatibilidad directa con tipos SQL.
- Simplicidad de serialización/deserialización.
- Bajo costo de memoria.
- Claridad semántica.

No se introducen tipos complejos innecesarios, lo que favorece mantenibilidad.

2 Identificador Entero (ID)

El uso de un identificador entero responde a criterios de:

- Eficiencia en indexación.
- Compatibilidad con auto-increment en SQL Server.
- Rapidez en búsquedas por clave primaria.

El ID funciona como punto de referencia en operaciones como:

- GET /api/libros/{id}
- PUT /api/libros/{id}
- DELETE /api/libros/{id}
- PATCH /api/libros/{id}/disponible

3 Campo Disponible como Booleano

La inclusión del atributo Disponible no es trivial; permite implementar una funcionalidad específica sin eliminar registros físicamente.

Ventajas:

- Evita borrado lógico innecesario.
- Permite control dinámico de estado.
- Facilita implementación del endpoint PATCH.
- Mejora trazabilidad del catálogo.

Esta decisión muestra que el modelo fue diseñado pensando en comportamiento del sistema, no solo en almacenamiento.

Relación del Modelo con la Base de Datos

El modelo Libro mantiene una correspondencia directa con la tabla Libros en SQL Server.

Relación conceptual:

Campo SQL	Atributo Modelo
id	ID
titulo	Titulo
autor	Autor
anio	Año
disponible	Disponible

Esta alineación directa simplifica:

- Inserciones.
- Actualizaciones.
- Escaneo de filas (rows.Scan()).
- Mantenimiento del código.

Jhonnatan\SQL... - dbo.Libros

Id	Titulo	Autor	Categoria	Anio	Formato	UrlArchivo	Disponible	CreatedAt	UpdatedAt
1	Cien años de so...	Gabriel García ...	Novela	1967	EPUB	cien_anos_de_s...	True	2026-03-02 01:2...	2026-03-02 01:2...
2	Don Quijote de ...	Miguel de Cerv...	Clásico	1605	PDF	don_quijote.pdf	True	2026-03-02 01:2...	2026-03-02 01:2...
3	La Odisea	Homero	Épico	-700	PDF	la_odisea.pdf	True	2026-03-02 01:2...	2026-03-02 01:2...
4	El Principito	Antoine de Sain...	Literatura	1943	PDF	el_principito.pdf	True	2026-03-02 01:2...	2026-03-02 01:2...
5	1984	George Orwell	Ciencia ficción	1949	EPUB	1984.epub	True	2026-03-02 01:2...	2026-03-02 01:2...
6	Rebelión en la ...	George Orwell	Política	1945	PDF	rebelion_granja...	True	2026-03-02 01:2...	2026-03-02 01:2...
7	Crimen y castigo	Fiódor Dostoye...	Novela	1866	EPUB	crimen_castigo....	False	2026-03-02 01:2...	2026-03-02 01:2...
8	El señor de los ...	J. R. R. Tolkien	Fantasía	1954	PDF	senor_anillos.pdf	True	2026-03-02 01:2...	2026-03-02 01:2...
9	El hobbit	J. R. R. Tolkien	Fantasía	1937	EPUB	el_hobbit.epub	True	2026-03-02 01:2...	2026-03-02 01:2...
10	Fahrenheit 451	Ray Bradbury	Ciencia ficción	1953	PDF	fahrenheit_451....	True	2026-03-02 01:2...	2026-03-02 01:2...
11	Drácula	Bram Stoker	Terror	1897	EPUB	dracula.epub	False	2026-03-02 01:2...	2026-03-02 01:2...
12	Frankenstein	Mary Shelley	Terror	1818	PDF	frankenstein.pdf	True	2026-03-02 01:2...	2026-03-02 01:2...
13	Harry Potter y l...	J. K. Rowling	Fantasía	1997	EPUB	harry_potter_1....	True	2026-03-02 01:2...	2026-03-02 01:2...
14	El código Da Vi...	Dan Brown	Suspenso	2003	PDF	codigo_da_vinc...	False	2026-03-02 01:2...	2026-03-02 01:2...
15	Los juegos del ...	Suzanne Collins	Distopía	2008	EPUB	juegos_hambre...	True	2026-03-02 01:2...	2026-03-02 01:2...
16	La metamorfosis	Franz Kafka	Literatura	1915	PDF	metamorfosis.p...	True	2026-03-02 01:2...	2026-03-02 01:2...
17	Orgullo y prejui...	Jane Austen	Romance	1813	EPUB	orgullo_prejuici...	False	2026-03-02 01:2...	2026-03-02 01:2...
18	El alquimista	Paulo Coelho	Ficción	1988	PDF	el_alquimista.pdf	True	2026-03-02 01:2...	2026-03-02 01:2...
19	Ensayo sobre la...	José Saramago	Novela	1995	EPUB	ceguera.epub	True	2026-03-02 01:2...	2026-03-02 01:2...
20	El retrato de Do...	Oscar Wilde	Clásico	1890	PDF	dorian_gray.pdf	False	2026-03-02 01:2...	2026-03-02 01:2...
21	Guerra mundial Z	Max Brooks	Thriller	2015	EPUB	https://espaeb...	True	2026-03-02 01:2...	2026-03-02 01:2...
22	Abuelo cuenta...	Lidefonso Falco...	Biografías	2021	EPUB	https://espaeb...	True	2026-03-02 01:2...	2026-03-02 01:2...
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Ilustración 13 Tabla Libros en SQL Server

Independencia del Modelo frente a Infraestructura

Un aspecto clave en el diseño es que Libro:

- No importa paquetes de base de datos.
- No contiene lógica SQL.
- No depende de HTTP.
- No contiene reglas de negocio.

Esta pureza del modelo garantiza:

- Reutilización en otros contextos.
- Testabilidad independiente.
- Posibilidad de migrar infraestructura sin modificar dominio.

Este diseño está alineado con principios de arquitectura limpia y Domain-Centric Design.

Evaluación Técnica del Modelo

Desde un punto de vista profesional, el modelo cumple con:

- Claridad estructural.
- Correspondencia relacional coherente.
- Simplicidad semántica.
- Bajo acoplamiento.
- Preparación para extensibilidad.

Podría extenderse en el futuro con:

- Campos de auditoría (fecha_creacion, fecha_actualizacion).
- Identificadores UUID.
- Validaciones estructurales.
- Métodos asociados.

Sin embargo, para el alcance actual del proyecto, el diseño es consistente y suficiente.

Descripción de Repository

Rol del Repository dentro de la Arquitectura

La capa **Repository** constituye el mecanismo formal mediante el cual la lógica de negocio interactúa con el sistema de persistencia sin depender directamente de la tecnología subyacente. En términos de diseño, el repositorio actúa como un **puente entre el dominio y la infraestructura**, encapsulando el acceso a datos y evitando que la capa de negocio contenga instrucciones SQL o detalles de conexión.

En este proyecto, el repositorio está compuesto por dos elementos claramente diferenciados:

1. **Interfaz BookRepository** (Contrato abstracto).
2. **Implementación concreta SQLServerStore** (Infraestructura real).

Esta separación responde al principio de **Inversión de Dependencias**, donde las capas superiores (Catalog) dependen de abstracciones y no de implementaciones concretas.

Interfaz BookRepository (Contrato de Persistencia)

La interfaz BookRepository define el conjunto de operaciones que el sistema requiere para gestionar libros. No contiene lógica ni implementación; únicamente declara métodos.

Ejemplo conceptual:

```
type BookRepository interface {
   AddBook(libro Libro) error
   GetByID(id int) (Libro, error)
   UpdateBook(libro Libro) error
   DeleteBook(id int) error
   SearchBooks(query string) ([]Libro, error)
   ToggleAvailability(id int) error
}
```

Análisis Técnico

Esta interfaz cumple funciones críticas:

- Define el **contrato mínimo** necesario para el funcionamiento del sistema.
- Permite que la lógica del catálogo sea independiente del motor de base de datos.
- Facilita pruebas unitarias mediante repositorios simulados.
- Permite intercambiar la persistencia sin modificar la lógica de negocio.

Desde el punto de vista arquitectónico, BookRepository representa una abstracción de almacenamiento, no una implementación específica.

Justificación del Diseño Basado en Interfaces

La decisión de utilizar una interfaz responde a principios formales de diseño orientado a objetos:

- **Programar contra abstracciones.**
- **Reducir acoplamiento.**
- **Facilitar extensibilidad.**
- **Habilitar testabilidad.**

Si el sistema dependiera directamente de SQLServerStore, cualquier cambio en la base de datos impactaría en la lógica del negocio. Al depender de BookRepository, el catálogo permanece estable incluso si la persistencia cambia.

Representación conceptual:

Catalog → BookRepository ← SQLServerStore

Este esquema demuestra que la implementación concreta está “debajo” del contrato, no por encima.

Implementación Concreta: SQLServerStore

La clase SQLServerStore implementa la interfaz BookRepository utilizando SQL Server como motor de persistencia.

Responsabilidades principales:

- Establecer conexión con la base de datos.
- Ejecutar sentencias SQL (INSERT, SELECT, UPDATE, DELETE).
- Mapear filas a estructuras Libro.
- Manejar errores de infraestructura.
- Garantizar consistencia de datos.

Ejemplo conceptual simplificado:

```
type SQLServerStore struct {
   db *sql.DB
}
```

Y un método representativo:

```
func (s *SQLServerStore) AddBook(libro Libro) error {
   query := "INSERT INTO Libros (...) VALUES (...)"
   _, err := s.db.Exec(query, ...)
   return err
}
```

Manejo de Errores en el Repository

La capa de repositorio es responsable de propagar errores técnicos hacia la capa superior. Esto permite que:

- La API retorne códigos HTTP adecuados.
- El catálogo pueda reaccionar ante fallos.
- Se mantenga separación entre errores técnicos y reglas de negocio.

Ejemplos de errores manejados:

- Fallos de conexión.
- Registros no encontrados.
- Violaciones de clave primaria.
- Problemas de ejecución SQL.

La decisión de retornar error en todos los métodos es consistente con las convenciones idiomáticas de Go.

Descripción de Catalog

Rol del Catalog dentro del Sistema

La capa **Catalog** representa el núcleo funcional del sistema, ya que centraliza la **lógica de negocio** y orquesta el uso del repositorio sin exponer detalles de infraestructura. En términos de arquitectura, Catalog actúa como el componente que "interpreta" lo que significa gestionar un libro electrónico dentro del dominio del proyecto. Esto lo diferencia de la API, que únicamente transporta solicitudes HTTP, y del repositorio, que únicamente persiste datos.

En un sistema correctamente diseñado, la API no debe contener reglas de negocio (por ejemplo: validaciones, consistencia, decisiones de estado), y la base de datos tampoco debe convertirse en el lugar donde se "resuelven" políticas del dominio. El Catalog existe precisamente para evitar esa dispersión y garantizar que el sistema tenga una fuente única de verdad para su comportamiento.

A nivel técnico, el catálogo se caracteriza por:

- Depender de una **interfaz** (BookRepository) y no de una implementación concreta.
- Concentrar operaciones del dominio de forma consistente.
- Permitir pruebas unitarias aisladas (sin necesidad de levantar el servidor web).
- Facilitar evolución futura (agregar auditoría, cache, validaciones avanzadas o seguridad) sin reestructurar el sistema.

Diseño Interno y Control de Responsabilidades

El catálogo debe ser interpretado como una capa que combina dos funciones:

1. **Orquestación:** decide qué operación ejecutar y con qué parámetros.
2. **Reglas del dominio:** valida consistencia y controla estados permitidos.

Este diseño es especialmente importante en sistemas CRUD, porque un error común es que el CRUD se ejecute como simple "pasamanos" hacia la base de datos. En un proyecto integrador de alta complejidad, se espera que exista una capa que:

- Controle integridad lógica antes de persistir.
- Garantice que las actualizaciones no dejen el sistema en estados inconsistentes.
- Centralice decisiones sobre disponibilidad, búsqueda y cambios.

La capa Catalog, por tanto, es la materialización de la POO en el backend: encapsula comportamiento y estructura, no solo datos.

Métodos Principales del Catálogo

AddBook (Crear un Libro)

AddBook representa la operación de incorporación de un nuevo libro al sistema. Su responsabilidad no es únicamente “insertar” un registro, sino garantizar que el objeto Libro cumpla condiciones mínimas de validez antes de ser persistido.

En una implementación profesional, esta operación suele involucrar:

- Verificar que campos obligatorios no estén vacíos (Titulo, Autor).
- Validar que Año esté dentro de rangos razonables.
- Definir un estado inicial coherente (por ejemplo, Disponible = true por defecto si aplica).
- Delegar el almacenamiento al repositorio.

En términos de arquitectura, AddBook evidencia el principio de que el catálogo controla el dominio, y la persistencia es una consecuencia, no el centro del sistema.

GetByID (Consultar un Libro por Identificador)

Este método implementa una operación fundamental en catálogos: recuperar un recurso de forma directa y determinística mediante su clave primaria.

Responsabilidades clave:

- Validar que el ID sea válido (no negativo, no cero si se define así).
- Delegar la recuperación al repositorio.
- Retornar el resultado a la capa superior sin exponer detalles de consulta.

En sistemas REST, esta operación se refleja directamente en endpoints como:

- GET /api/libros/{id}

El método GetByID no solo habilita consulta, sino que normaliza el acceso a libros en todas las funcionalidades que dependen de un ID.

SearchBooks (Búsqueda)

SearchBooks introduce una funcionalidad adicional sobre el CRUD tradicional, elevando el nivel del sistema al incorporar consultas por criterios. Esto es importante porque muestra que el sistema no solo almacena, sino que permite recuperar información bajo necesidades prácticas (filtrado por título, autor u otros campos).

Este método debe interpretarse como un “servicio de consulta” donde:

- Se recibe un término de búsqueda.
- Se normaliza o valida el parámetro (evitar búsquedas vacías o innecesarias).
- Se delega al repositorio la ejecución de búsqueda.
- Se retorna una lista de coincidencias.

REST asociado:

- GET /api/libros/buscar?query=...

UpdateBook (Actualizar)

UpdateBook representa una operación crítica porque actualiza estado persistente y debe garantizar integridad. En proyectos reales, la actualización suele implicar validación de existencia previa y consistencia del objeto antes de persistir.

Responsabilidades típicas:

- Validar que el ID exista o que el libro tenga ID válido.
- Evitar que la actualización deje atributos en estado inválido.
- Preservar campos que no deban perderse (por ejemplo disponibilidad si no se actualiza desde allí).
- Delegar la actualización al repositorio.

REST asociado:

- PUT /api/libros/{id}

DeleteBook (Eliminar)

La operación de eliminación se justifica porque en catálogos el mantenimiento requiere remover recursos, ya sea por depuración o eliminación de registros inválidos.

Responsabilidades:

- Verificar que el ID sea válido.
- Delegar eliminación al repositorio.
- Propagar errores de “no encontrado” o fallos SQL.

REST asociado:

- DELETE /api/libros/{id}

Desde un enfoque técnico, este método permite evidenciar control completo del ciclo de vida de los objetos dentro del sistema.

ToggleAvailability (Cambiar Disponibilidad)

Esta operación representa una funcionalidad específica que incrementa el valor del sistema, porque introduce un cambio de estado parcial sin requerir reemplazo completo del recurso.

Su aporte técnico es doble:

1. **Introduce un comportamiento de negocio real**, donde un libro puede estar disponible o no.
2. Permite demostrar uso de un endpoint semánticamente correcto: **PATCH**.

REST asociado:

- PATCH /api/libros/{id}/disponible

Este método es una evidencia de que el sistema no se limita a CRUD básico, sino que modela un comportamiento útil y coherente.

Descripción de API

La API constituye la **interfaz pública del sistema**, encargada de traducir solicitudes HTTP en llamadas a la lógica de negocio (Catalog) y devolver respuestas estructuradas al cliente. Arquitectónicamente, actúa como una **capa de adaptación de transporte**, sin implementar reglas de negocio ni ejecutar consultas SQL directamente.

La API está implementada mediante:

- Handlers HTTP.
- Configuración de rutas en el router.
- Uso del paquete net/http.
- Serialización y deserialización estructurada.
- Manejo explícito de códigos de estado HTTP.

El sistema implementa más de ocho endpoints REST, incluyendo operaciones CRUD completas y funcionalidades adicionales como búsqueda y estadísticas.

Se respeta la semántica HTTP:

- GET → Lectura
- POST → Creación
- PUT → Actualización completa

- DELETE → Eliminación
- PATCH → Actualización parcial

La serialización estructurada permite interoperabilidad entre el backend y múltiples clientes (frontend HTML, Postman, futuras aplicaciones móviles). Asimismo, el manejo adecuado de códigos de estado (200, 201, 400, 404, 500) garantiza claridad y cumplimiento de estándares REST.

En términos arquitectónicos, la API mantiene separación estricta de responsabilidades: recibe datos, invoca el catálogo y retorna resultados sin contener lógica del dominio.

Concurrencia

El servidor HTTP en Go maneja cada solicitud en una **goroutine independiente**, lo que introduce un entorno inherentemente concurrente. Sin mecanismos de control, múltiples operaciones simultáneas podrían generar condiciones de carrera y corrupción del estado.

Para garantizar consistencia, el sistema implementa sync.RWMutex, permitiendo:

- Bloqueos de lectura simultánea (RLock).
- Bloqueos exclusivos de escritura (Lock).

Este enfoque optimiza rendimiento, ya que múltiples consultas GET pueden ejecutarse en paralelo, mientras que las operaciones de modificación (POST, PUT, DELETE, PATCH) se protegen de manera exclusiva.

La implementación previene:

- Escrituras conflictivas.
- Actualizaciones simultáneas inconsistentes.
- Eliminaciones concurrentes problemáticas.

Además, este control a nivel de aplicación complementa el control transaccional de SQL Server, fortaleciendo la robustez general del sistema.

La inclusión explícita de sincronización demuestra dominio de programación concurrente y eleva el nivel técnico del proyecto más allá de un CRUD básico.

Base de Datos

La persistencia del sistema se implementa en **SQL Server**, garantizando durabilidad, integridad estructural y consistencia transaccional.

La tabla principal Libros se define con:

- id (clave primaria autoincremental)
- titulo (texto)
- autor (texto)
- anio (entero)
- disponible (booleano)

Existe correspondencia directa entre el modelo Libro en Go y la tabla relacional, lo que simplifica el mapeo y reduce complejidad en la capa de persistencia.

El uso de consultas parametrizadas mejora seguridad y previene inyección SQL.

Asimismo, la estructura relacional permite futura expansión mediante índices, relaciones adicionales y campos de auditoría.

La base de datos está encapsulada dentro de la capa storage, evitando que el resto del sistema dependa directamente del motor SQL.

Desde la perspectiva académica y profesional, esta implementación demuestra integración real con un sistema gestor empresarial y garantiza que el estado del sistema sea persistente entre ejecuciones.

Pruebas Realizadas

La validación del **Sistema de Gestión de Libros Electrónicos** se realizó mediante una estrategia escalonada que incluye:

1. **Pruebas Unitarias**
2. **Pruebas de Integración**
3. **Pruebas de Aceptación**

Este enfoque permite verificar el sistema desde lo más interno (lógica aislada) hasta el flujo completo visible por el usuario final.

Pruebas Unitarias

Las pruebas unitarias se enfocan en validar el comportamiento de la capa Catalog de manera aislada, sin depender del servidor HTTP ni del entorno completo de ejecución.

Se ejecutan mediante el comando:

```
PS C:\Users\jhonn\OneDrive\Documents\Antigravity\Sistema_Gestion_Libros_Electronicos> go test ./... -v
?       sistema_libros/cmd/app  [no test files]
?       sistema_libros/cmd/server       [no test files]
?       sistema_libros/internal/api     [no test files]
=== RUN   TestGetByID_Found
--- PASS: TestGetByID_Found (0.00s)
=== RUN   TestGetByID_NotFound
--- PASS: TestGetByID_NotFound (0.00s)
=== RUN   TestSearchBooks_ByTitle
--- PASS: TestSearchBooks_ByTitle (0.00s)
=== RUN   TestSearchBooks_ByAuthor
--- PASS: TestSearchBooks_ByAuthor (0.00s)
=== RUN   TestSearchBooks_ByCategory
--- PASS: TestSearchBooks_ByCategory (0.00s)
=== RUN   TestAddBook_AssignsNewID
--- PASS: TestAddBook_AssignsNewID (0.00s)
=== RUN   TestUpdateBook_UpdatesFields
--- PASS: TestUpdateBook_UpdatesFields (0.00s)
=== RUN   TestUpdateBook_NotFound
--- PASS: TestGetByID_NotFound (0.00s)
=== RUN   TestSearchBooks_ByTitle
--- PASS: TestSearchBooks_ByTitle (0.00s)
=== RUN   TestSearchBooks_ByAuthor
--- PASS: TestSearchBooks_ByAuthor (0.00s)
=== RUN   TestSearchBooks_ByCategory
--- PASS: TestSearchBooks_ByCategory (0.00s)
=== RUN   TestAddBook_AssignsNewID
--- PASS: TestAddBook_AssignsNewID (0.00s)
=== RUN   TestUpdateBook_UpdatesFields
```

Objetivo

- Verificar que los métodos del catálogo cumplen su comportamiento esperado.
- Detectar errores lógicos.
- Validar reglas del dominio.
- Prevenir regresiones futuras.

Casos Cubiertos

Las pruebas incluyen validaciones como:

- Agregar un libro correctamente.
- Obtener libro por ID.
- Buscar libros por criterio.
- Actualizar libro.
- Eliminar libro.
- Cambiar disponibilidad.

Tabla resumen:

Método	Caso Validado
AddBook	Inserción correcta
GetByID	Recuperación válida
UpdateBook	Modificación persistente
DeleteBook	Eliminación correcta
ToggleAvailability	Cambio de estado

Pruebas de Integración

Las pruebas de integración verifican la interacción entre:

- API REST
- Catalog
- SQLServerStore
- Base de datos SQL Server

Aquí se valida que las operaciones realizadas a través de HTTP impacten correctamente en la base de datos.

Herramientas utilizadas

- Navegador
- Postman
- curl

Ejemplos de Validación

1. Enviar POST /api/libros
2. Verificar en SSMS que el registro fue insertado.
3. Enviar PUT /api/libros/{id}
4. Confirmar actualización en base de datos.
5. Enviar DELETE /api/libros/{id}
6. Confirmar eliminación en SQL Server.

Este tipo de pruebas confirma que la integración entre capas funciona correctamente.

```
Problems    Output    Debug Console    Terminal    Ports

PS C:\Users\jhonn\OneDrive\Documents\Antigravity\Sistema_Gestion_Libros_Electronicos> go run ./cmd/server
====================================
Servidor:  http://localhost:8080/
API:       http://localhost:8080/api/libros
Health:    http://localhost:8080/health
====================================
GET / (510.9µs)
GET /api/libros (17.4359ms)
POST /api/libros (123.1015ms)
GET /api/libros (0s)
```

Ilustración 14 Libro añadido

Pruebas de Aceptación

Las pruebas de aceptación validan el funcionamiento completo desde la perspectiva del usuario final.

Se ejecutan accediendo a:

http://localhost:8080/

Y realizando acciones mediante la interfaz web.

Flujo Validado

1. Ingresar datos en el formulario.

2. Crear libro.
3. Visualizar listado actualizado.
4. Buscar libro.
5. Cambiar disponibilidad.
6. Eliminar libro.

Estas pruebas confirman que:

- El frontend consume correctamente la API.
- Las respuestas se reflejan visualmente.
- El sistema funciona extremo a extremo.

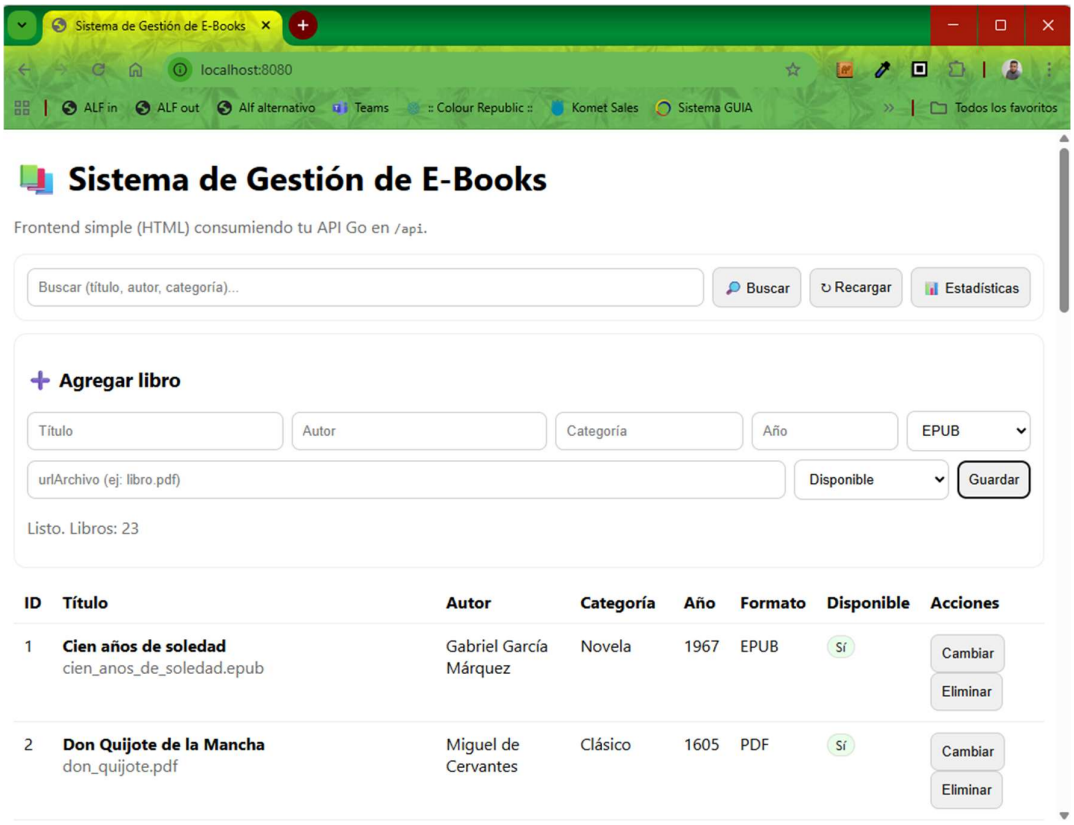

Ilustración 15 Interfaz Web

Dificultades Encontradas

El desarrollo del **Sistema de Gestión de Libros Electrónicos** implicó enfrentar diversos desafíos técnicos y conceptuales que permitieron consolidar conocimientos en programación orientada a objetos, arquitectura backend y persistencia en bases de datos relacionales.

Configuración y Conectividad con SQL Server

Uno de los principales desafíos fue la correcta configuración de la conexión con SQL Server. Problemas como:

- Instancia incorrecta del servidor.
- Uso del puerto equivocado.
- Configuración del SQL Browser.
- Manejo de cadenas de conexión.

Generaron errores iniciales de conexión que requirieron:

- Ajustar la dirección a localhost,1433.
- Verificar credenciales.
- Validar configuración del servicio SQL Server.

Este proceso permitió comprender mejor la importancia de la configuración de infraestructura en sistemas reales.

Manejo Correcto de Rutas y Servidor HTTP

Durante la implementación de la API y la integración con el frontend, surgieron errores como:

- Respuestas 404 al intentar servir el archivo HTML.
- Conflictos en la configuración del router.
- Problemas en el uso de http.FileServer.

La solución implicó reorganizar el mux principal y garantizar que las rutas estuvieran correctamente registradas antes de iniciar el servidor.

Esta dificultad permitió comprender mejor el flujo interno de net/http y la importancia del orden en la configuración del servidor.

Sincronización y Concurrencia

La implementación de concurrencia mediante sync.RWMutex representó un desafío conceptual importante. Entender:

- Diferencia entre Lock() y RLock().
- Escenarios de condiciones de carrera.
- Impacto en rendimiento.
- Alcance de los bloqueos.

Requirió análisis profundo del modelo concurrente en Go.

Este aprendizaje fue clave para asegurar que el sistema pudiera manejar múltiples solicitudes simultáneamente sin comprometer integridad.

Separación Correcta de Responsabilidades

Otro reto fue evitar que la lógica de negocio se mezclara con:

- Handlers HTTP.
- Consultas SQL.
- Código de infraestructura.

Fue necesario reorganizar el proyecto en capas claras (model, catalog, storage, api) para mantener coherencia arquitectónica.

Este proceso fortaleció la comprensión práctica del principio de **Separación de Responsabilidades** y del uso adecuado de interfaces.

Corrección de Pruebas Unitarias

Durante la implementación de pruebas, se detectaron errores relacionados con:

- Actualización incorrecta del campo Disponible.
- Inconsistencias en métodos de actualización.

Estos errores obligaron a revisar la lógica interna del catálogo, mejorando la robustez del sistema.

Reflexión Técnica

Las dificultades encontradas no representaron únicamente obstáculos, sino oportunidades de aprendizaje en:

- Arquitectura backend.
- Manejo de base de datos empresarial.
- Programación concurrente.
- Diseño orientado a interfaces.
- Depuración estructurada.

Estas experiencias fortalecieron la calidad final del proyecto y permitieron consolidar conocimientos aplicados.

Conclusiones

Durante el desarrollo del Sistema de Gestión de Libros Electrónicos logré aplicar de manera integral los principios de Programación Orientada a Objetos en un entorno real, comprendiendo que la arquitectura del sistema es tan importante como su

funcionamiento. A través de la separación por capas, el uso de interfaces y la implementación del patrón Repository, entendí cómo reducir el acoplamiento y diseñar soluciones escalables y mantenibles. Este proyecto me permitió pasar de una comprensión teórica de la POO a una aplicación práctica estructurada y profesional.

La integración de una API REST conectada a SQL Server me permitió experimentar el ciclo completo de desarrollo backend, desde la exposición de servicios hasta la persistencia de datos en un motor relacional empresarial. Comprendí la importancia del uso correcto de métodos HTTP, el manejo adecuado de códigos de estado y la necesidad de encapsular la lógica de negocio fuera de la capa de transporte. Además, la implementación de concurrencia mediante sync.RWMutex fortaleció mi entendimiento del comportamiento concurrente en Go y la importancia de proteger recursos compartidos.

Este proyecto no solo cumplió con los requisitos académicos, sino que representó una experiencia de aprendizaje profundo en arquitectura, diseño estructurado y pruebas de software. Aprendí a enfrentar dificultades técnicas reales, como problemas de conexión, manejo de rutas y sincronización, y a resolverlas mediante análisis y depuración sistemática. Considero que el sistema desarrollado constituye una base sólida para futuras mejoras, como autenticación, despliegue en la nube y microservicios, demostrando que fue diseñado con visión de crecimiento y profesionalismo.

Bibliografía

Erickson, J. (4 de abril de 2024). *Oracle*. Obtenido de Oracle: https://www.oracle.com/latam/database/what-is-json/#:~:text=din%C3%A1micas%20e%20interactivas.-,%C2%BFPor%20qu%C3%A9%20se%20utiliza%20JSON?,necesidad%20de%20cambios%20de%20c%C3%B3digo.

Herrera, J. (3 de enero de 2024). *Medium*. Obtenido de Medium: https://medium.com/@jaime.herrera.mz/orientaci%C3%B3n-a-objetos-en-go-b74a3a2a5a58

Ramirez, O. (29 de julio de 2024). *Bambu Mobile*. Obtenido de Bambu Mobile: https://bambu-mobile.com/ciclo-de-desarrollo-de-software/#:~:text=hasta%20el%20mantenimiento.-,Planificaci%C3%B3n,desarrollar%20y%20c%C3%B3mo%20se%20lograr%C3%A1.

URL web repositorio de Github

https://github.com/Jhonna555/Jhonna555-Sistema_de_Gestion_de_libros_Electronicos.git

Nota Final

Todos los archivos correspondientes al proyecto, incluyendo:

- Código fuente completo del sistema.
- Estructura del proyecto organizada por capas.
- Video demostrativo del funcionamiento.

Se encuentran disponibles en el repositorio oficial de GitHub del proyecto.